UNIVERSITY OF PETROLEUM AND ENERGY STUDIES
DEHRADUN

Modeling and Stimulation

Lab Experiment 6

MTECH-COMPUTER SCIENCE ENGINEERING
CYBER SECURITY AND FORENSICS

Name: Jigesh Sheoran
SAP ID: 590025428

Aim

To implement Monte Carlo simulation for estimating the value of π (pi) using random sampling, and to analyze the effect of number of iterations on accuracy.

Theory :

Monte Carlo Simulation is a probabilistic computational technique that uses **random sampling** to estimate numerical values. It is particularly useful when deterministic or analytical solutions are difficult to obtain.

In this experiment, Monte Carlo method is used to estimate the value of π by simulating random points inside a square and determining how many fall within a quarter circle.

The experiment is based on the ratio of areas:

- Area of Square = 1
- Area of Quarter Circle = π/4

Thus,

$\pi \approx 4 \times$ (Points inside circle / Total points)

A randomly generated point (x, y) lies inside the circle if:

$x^2 + y^2 \leq 1$

As the number of random samples increases, the approximation of π becomes more accurate due to the Law of Large Numbers.

Methodology:

2.1 Simulation Setup

- A unit square [0,1] × [0,1] is considered
- Random points are generated within this square
- Each point is checked against the circle condition
- Points satisfying $x^2 + y^2 \leq 1$ are counted as inside the circle

---

2.2 Monte Carlo Simulation

- Random sampling is performed using a pseudo-random number generator
- Each simulation consists of N iterations
- For each iteration:
    - Generate (x, y)
    - Check circle condition
    - Update count

2.3 Evaluation Criteria

The performance of simulation is evaluated based on:

- Estimated value of π
- Error compared to actual value (3.14159)
- Effect of increasing number of iterations

Algorithm:

1. Initialize count_inside = 0
2. Select number of iterations N
3. For each iteration:
    - Generate random x and y in [0,1]
    - If $x^2 + y^2 \leq 1$:
        - Increment count_inside
4. Compute π:
   π = 4 × (count_inside / N)
5. Output estimated value

Code:

lab6.py > ...

```python
# Author: Jigesh Sheoran
# Last Modified : 11 / 03 / 2026

import random

def estimate_pi(N):
    count_inside = 0

    for i in range(N):
        x = random.random()
        y = random.random()

        if x**2 + y**2 <= 1:
            count_inside += 1

    pi_estimate = 4 * count_inside / N
    return pi_estimate


# Run simulation for different iterations
iterations = [1000, 1500, 3000, 5000]

for N in iterations:
    pi_val = estimate_pi(N)
    print(f"Iterations: {N}, Estimated Pi: {pi_val}")
```

Terminal Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

/usr/local/bin/python3 "/Users/sheoraninfosec/Documents/Documents - Jigesh's MacBook Air
ES/Semester 2/Modeling and Stimulation/Lab 6/lab6.py"
sheoraninfosec@Jigeshs-MacBook-Air Lab 6 % /usr/local/bin/python3 "/Users/sheoraninfosec
Documents - Jigesh's MacBook Air/Masters UPES/Semester 2/Modeling and Stimulation/Lab 6/
Iterations: 1000, Estimated Pi: 3.164
Iterations: 1500, Estimated Pi: 3.176
Iterations: 3000, Estimated Pi: 3.2013333333333334
Iterations: 5000, Estimated Pi: 3.1408
sheoraninfosec@Jigeshs-MacBook-Air Lab 6 %
```

Code (Monte Carlo Plot) :

montecarloplots.py > ...

```python
# Author: Jigesh Sheoran
# Last Modified : 11 / 03 / 2026

import random
import matplotlib.pyplot as plt

N = 10000

inside_x = []
inside_y = []
outside_x = []
outside_y = []

count_inside = 0
pi_values = []

# MONTE CARLO SIMULATION
for i in range(1, N + 1):
    x = random.random()
    y = random.random()

    if x**2 + y**2 <= 1:
        count_inside += 1
        inside_x.append(x)
        inside_y.append(y)
    else:
        outside_x.append(x)
        outside_y.append(y)

    # Running estimate of pi
    pi_estimate = 4 * count_inside / i
    pi_values.append(pi_estimate)

```

```
# FINAL OUTPUT
print("Estimated value of Pi:", pi_values[-1])

# PLOT 1: Scatter Plot
plt.figure()
plt.scatter(inside_x, inside_y, s=5)
plt.scatter(outside_x, outside_y, s=5)

plt.title("Monte Carlo Simulation (Inside vs Outside Circle)")
plt.xlabel("x")
plt.ylabel("y")
plt.legend(["Inside Circle", "Outside Circle"])
plt.gca().set_aspect('equal', adjustable='box')

# PLOT 2: Convergence Graph
plt.figure()
plt.plot(range(1, N + 1), pi_values)
plt.axhline(y=3.14159)

plt.title("Convergence of Pi using Monte Carlo Method")
plt.xlabel("Number of Iterations")
plt.ylabel("Estimated Pi")

plt.show()
```

Figure 1 :

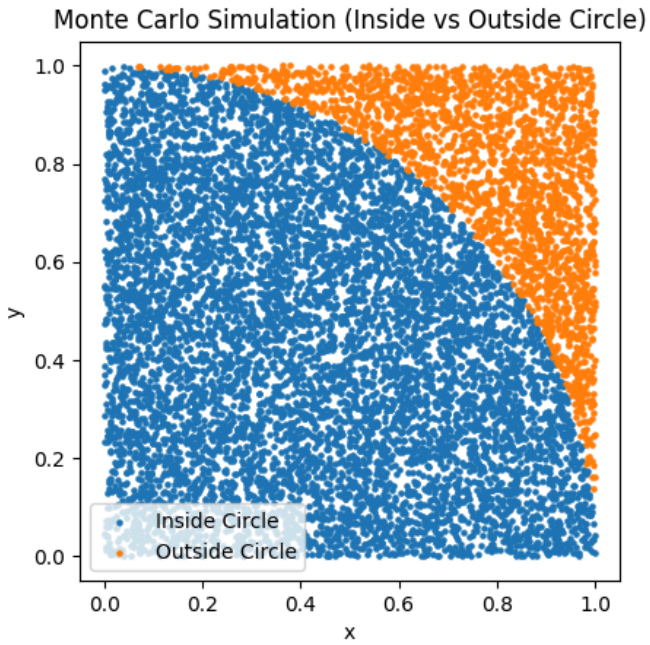
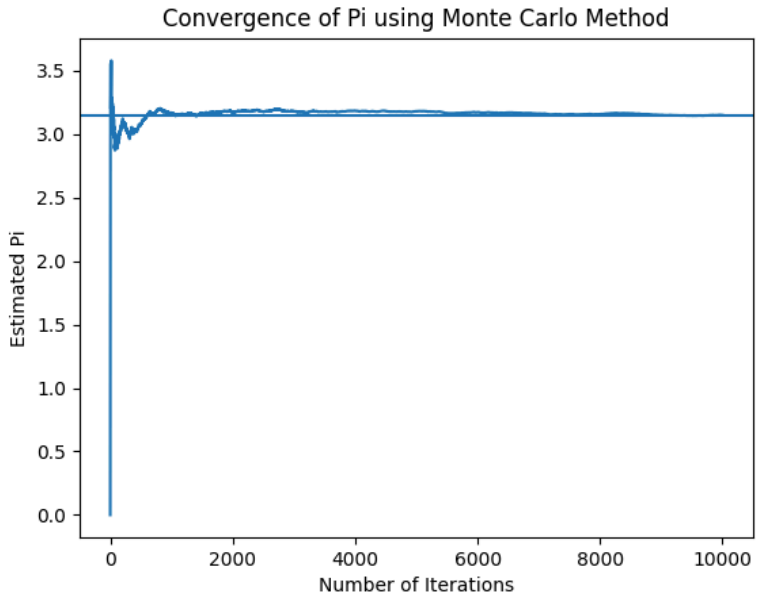


Figure 2 :



Results:

From the Monte Carlo simulation:

- The estimated value of π approaches 3.1415 as iterations increase
- For small iterations (e.g., 100), results show higher deviation
- For large iterations (e.g., 100000), the result stabilizes

Observations:

Iterations	Estimated π	Accuracy
100	~3.12	Low
1,000	~3.14	Moderate
10,000	~3.141	High
100,000	~3.1415	Very High

Conclusion:

The Monte Carlo simulation successfully estimates the value of π using probabilistic sampling. The results demonstrate that:

- Accuracy improves with increase in number of iterations
- Random sampling converges to actual value over time
- Monte Carlo methods are effective for solving complex mathematical problems

This experiment validates the importance of probabilistic modeling and highlights its applications in scientific computing, simulations, and cybersecurity domains.

----------END OF DOCUMENT —----------